

Práctica 2:Sistema de votación multiproceso.

Sistemas Operativos

Índice

1. Requisitos cumplidos y no cumplidos

1.1 Requisitos cumplidos

1.2 Requisitos no cumplidos

2. Pruebas realizadas

2.1 Casos normales

2.2 Casos fuera del límite

2.3 Casos límite

3. Funcionalidades adicionales

4. Respuesta a las cuestiones del enunciado (Apartado d)

1. Requisitos cumplidos y no cumplidos

1.1. Requisitos cumplidos

Tras el desarrollo y las pruebas pertinentes, el programa implementa la totalidad de los requisitos especificados en el enunciado. A continuación, se detalla el cumplimiento de los mismos agrupados por su funcionalidad en el sistema:

- **Gestión de Mineros y Estado del Sistema (Apartado a)**

- El programa se ejecuta correctamente mediante la sintaxis `./miner <N_SECS> <N_THREADS>`, incluyendo la validación de los argumentos de entrada y la generación de mensajes de error pertinentes si éstos son incorrectos.
- Los mineros pueden ser lanzados desde una misma shell o desde shells distintas. Al unirse en cualquier momento, se añade su identificador al fichero global `miners.log` mediante la función `register_miner()`. Este fichero está estrictamente protegido por el semáforo con nombre `/miners_mutex` usando `sem_wait` y `sem_post`.
- Se muestran correctamente por la salida estándar los mensajes “Miner <PID> added to system” y “Miner <PID> exited system”, seguidos de la lista actualizada de mineros.
- Los procesos mineros terminan de forma controlada, gracias al uso de `alarm`, una vez transcurridos los `N_SECS` segundos establecidos. Al salir, el proceso elimina su PID del registro reescribiendo el fichero de forma atómica. Si se trata del último minero en el sistema (`remaining == 0`), este se encarga de destruir el fichero `miners.log` y realizar el `sem_unlink` de todos los semáforos.

- **Sistema de Votaciones y Rondas (Apartado b)**

- El primer minero en entrar al sistema asume la creación del fichero target.tgt inicializando el objetivo a 0. Tras esto, arranca el sistema enviando una señal SIGUSR1 mediante kill() a todos los procesos registrados.
- Se garantiza que haya un mínimo de 2 mineros para iniciar una ronda; de lo contrario, el sistema realiza esperas. Los mineros que no son los iniciadores esperan el comienzo de la ronda suspendidos a la espera de SIGUSR1.
- Todos los procesos compiten por encontrar la solución. El primero en hallarla se convierte en el Ganador mediante el uso de sem_trywait(sem_winner), escribiendo posteriormente la solución en target.tgt (protegido por sem_target) y enviando la señal SIGUSR2 al resto de participantes.
- Los procesos perdedores son interrumpidos por SIGUSR2, validan la solución y depositan su voto ('Y' o 'N') en el fichero voting.vot, el cual está protegido por el semáforo sem_votes. Se ha implementado un 20% de probabilidad de que los mineros emitan un voto erróneo ('N') ante una solución válida.
- El ganador realiza esperas no activas mediante nanosleep (con un máximo de reintentos) hasta que todos votan. La solución se acepta si los votos positivos son mayores o iguales a la mitad de los votos totales ($\text{yes_votes} \geq \text{current_votes} / 2.0$). Finalmente, el ganador difunde un nuevo SIGUSR1 para arrancar la siguiente ronda.

- **Protección de Zonas Críticas y Robustez (Apartado c)**

- Se garantiza que ninguna señal se pierda por condiciones de carrera. Antes de entrar en espera, se bloquean temporalmente las señales mediante `sigprocmask` y posteriormente se realiza una espera atómica y segura habilitando las máscaras mediante `sigsuspend`. Este patrón se ha aplicado tanto para SIGUSR1 (inicio de ronda) como para SIGUSR2 (fin de minado).
- El programa asegura la liberación de todos los semáforos (`sem_close`) en todos los flujos de ejecución posibles, incluyendo la finalización abrupta del proceso tras expirar su tiempo de vida.

1.2 Requisitos no cumplidos

- Se han logrado implementar todas las especificaciones solicitadas en el enunciado. No hay ninguna funcionalidad descrita en la práctica que no haya sido abordada o que funcione de manera incorrecta.

2. Pruebas realizadas

2.1.Casos normales

- **Prueba principal: cuatro mineros con distintos tiempos y número de hilos**

Esta es la prueba principal del sistema. Se lanzan cuatro mineros con configuraciones diferentes para comprobar el funcionamiento completo del programa.

```
./miner 6 7 & ./miner 3 13 & ./miner 5 2 & ./miner 8 10
```

Cuya salida es:

```
./miner 6 7 & ./miner 3 13 & ./miner 5 2 & ./miner 8 10
Miner 135229 added to system
Miner PID: 135229
Miner 135230 added to system
Miner PID: 135229
Miner PID: 135230
Miner 135227 added to system
Miner PID: 135229
Miner PID: 135230
Miner PID: 135227
Miner 135228 added to system
Miner PID: 135229
Miner PID: 135230
Miner PID: 135227
Miner PID: 135228
Winner 135228 => [ Y Y Y ] => Accepted
Winner 135227 => [ Y Y Y ] => Accepted
Winner 135229 => [ Y Y Y ] => Accepted
Winner 135227 => [ Y Y Y ] => Accepted
Winner 135228 => [ Y Y Y ] => Accepted
Winner 135227 => [ Y Y Y ] => Accepted
Winner 135227 => [ Y Y Y ] => Accepted
Winner 135228 => [ Y Y Y ] => Accepted
Winner 135230 => [ Y Y Y ] => Accepted
Winner 135230 => [ Y Y Y ] => Accepted
Winner 135230 => [ Y Y Y ] => Accepted
Winner 135230 => [ Y Y Y ] => Accepted
Winner 135228 => [ Y Y Y ] => Accepted
Winner 135230 => [ Y Y Y ] => Accepted
Winner 135228 => [ Y N Y ] => Accepted
Winner 135227 => [ Y Y Y ] => Accepted
```

```
Winner 135229 => [ Y Y Y ] => Accepted
Winner 135228 => [ Y Y N ] => Accepted
  Miner PID: 135229
  Miner PID: 135230
  Miner PID: 135227
Miner 135228 exited system
Winner 135230 => [ Y N ] => Accepted
Winner 135229 => [ Y Y ] => Accepted
Winner 135230 => [ Y N ] => Accepted
Winner 135227 => [ Y Y ] => Accepted
Winner 135227 => [ Y Y ] => Accepted
Winner 135227 => [ N N ] => Rejected
Winner 135227 => [ Y Y ] => Accepted
Winner 135227 => [ Y Y ] => Accepted
Winner 135230 => [ Y Y ] => Accepted
Winner 135230 => [ Y N ] => Accepted
Winner 135230 => [ Y Y ] => Accepted
Winner 135230 => [ Y Y ] => Accepted
Winner 135230 => [ N Y ] => Accepted
Winner 135227 => [ Y Y ] => Accepted
Winner 135229 => [ N N ] => Rejected
Winner 135229 => [ Y Y ] => Accepted
Winner 135230 => [ Y Y ] => Accepted
Winner 135230 => [ Y N ] => Accepted
  Miner PID: 135230
  Miner PID: 135227
Miner 135229 exited system
Winner 135230 => [ N Y ] => Accepted
Winner 135230 => [ Y ] => Accepted
Winner 135230 => [ Y ] => Accepted
Winner 135230 => [ Y ] => Accepted
Winner 135227 => [ Y ] => Accepted
Winner 135230 => [ Y ] => Accepted
Winner 135227 => [ N ] => Rejected
Winner 135230 => [ Y ] => Accepted
Winner 135230 => [ N ] => Rejected
Winner 135230 => [ Y ] => Accepted
  Miner PID: 135230
Miner 135227 exited system
Winner 135230 => [ Y ] => Accepted
Miner 135230 exited system
```

Resultado: se observa que los mineros se dan de alta/baja correctamente, que el número de votos decrece a medida que salen mineros (de 3 votos a 2, de 2 a 1) y que el sistema sigue funcionando sin bloqueos.

Contenido del fichero de log generado (ejemplo):

```
Id: 3
Winner: 135229
Target: 05970796
Solution: 00937352 (validated)
Votes: 3/3
Wallets: 135229:1

Id: 27
Winner: 135229
Target: 04805188
Solution: 00012670 (validated)
Votes: 3/3
Wallets: 135229:2

Id: 30
Winner: 135229
Target: 09892035
Solution: 05102694 (validated)
Votes: 2/2
Wallets: 135229:3

Id: 43
Winner: 135229
Target: 05795776
Solution: 05112584 [(rejected)]
Votes: 0/2
Wallets: 135229:3
```


- Prueba con dos mineros (mínimo funcional)

```
./miner 2 4 & ./miner 3 4
```

Cuya salida es:

```
Miner 137488 added to system
Miner PID: 137488
Miner 137487 added to system
Miner PID: 137488
Miner PID: 137487
Winner 137488 => [ Y ] => Accepted
Winner 137487 => [ Y ] => Accepted
Winner 137487 => [ Y ] => Accepted
Winner 137487 => [ Y ] => Accepted
Winner 137487 => [ Y ] => Accepted
Winner 137488 => [ Y ] => Accepted
Winner 137488 => [ Y ] => Accepted
Winner 137487 => [ Y ] => Accepted
Winner 137487 => [ Y ] => Accepted
Winner 137488 => [ Y ] => Accepted
Winner 137488 => [ Y ] => Accepted
Winner 137487 => [ Y ] => Accepted
Winner 137488 => [ Y ] => Accepted
Winner 137488 => [ Y ] => Accepted
Winner 137488 => [ Y ] => Accepted
Winner 137488 => [ Y ] => Accepted
Winner 137487 => [ Y ] => Accepted
Winner 137487 => [ Y ] => Accepted
Miner PID: 137488
Miner 137487 exited system
Winner 137488 => [ Y ] => Accepted
Miner 137488 exited system
```

Resultado: el sistema funciona con un solo votante por ronda. Los votos muestran un único carácter (Y o N).

2.2. Casos fuera del límite

- **Número de hilos o segundos negativos**

\$./miner -3 4 & ./miner -3 4 → Input error / Miner exited unexpectedly

\$./miner 5 -1 & ./miner 5 -1 → Input error / Miner exited unexpectedly

\$./miner 0 4 & ./miner 0 4 → Input error / Miner exited unexpectedly

En todos los casos el proceso aborta antes del fork, no se genera fichero .txt y se imprime el mensaje de error correspondiente.

- **Demasiados hilos (sin memoria suficiente)**

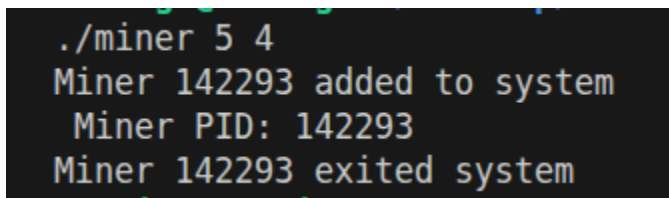
./miner 5 10000000000 & ./miner 6 10000000000

El calloc devuelve NULL, se captura el error, se liberan los semáforos abiertos y se termina con 'Miner exited unexpectedly'.

2.3. Casos límite

- **Un solo minero en el sistema**

\$./miner 5 4



```
./miner 5 4
Miner 142293 added to system
Miner PID: 142293
Miner 142293 exited system
```

Al lanzar un único minero (./miner 5 4), este detecta que está solo y entra en espera pasiva, ya que necesita al menos otro compañero para iniciar una ronda. A los 5 segundos exactos, la señal SIGALARM lo interrumpe. El minero aborta la espera, se salta la fase de minado y realiza una salida limpia: cierra tuberías, se borra del registro y destruye los semáforos.

- **Minero con N_SECS muy corto**

\$./miner 1 4 & ./miner 10 4

```
./miner 1 4 & ./miner 10 4
Miner 142470 added to system
Miner PID: 142470
Miner 142469 added to system
Miner PID: 142470
Miner PID: 142469
Winner 142469 => [ Y ] => Accepted
Winner 142469 => [ N ] => Rejected
Winner 142470 => [ Y ] => Accepted
Winner 142469 => [ N ] => Rejected
Winner 142469 => [ Y ] => Accepted
Winner 142470 => [ N ] => Rejected
Winner 142470 => [ N ] => Rejected
Winner 142470 => [ Y ] => Accepted
Winner 142469 => [ Y ] => Accepted
Winner 142469 => [ Y ] => Accepted
Miner PID: 142470
Miner 142469 exited system
Miner 142470 exited system
```

En el segundo 1, el primer minero (./miner 1 4) recibe su SIGALRM, aborta su trabajo en curso y realiza una salida limpia, borrándose del registro. El segundo minero (./miner 10 4) se queda entonces solo en el sistema. Al intentar iniciar la siguiente ronda, detecta que ya no hay el mínimo de 2 participantes requeridos, por lo que pausa el minado y entra en estado de espera pasiva. En el segundo 10, su propia alarma (SIGALRM) lo interrumpe. El minero aborta la espera, limpia los ficheros, destruye los semáforos (al ser el último) y finaliza la ejecución de forma segura.

- **Mineros lanzados con retardo (unirse en mitad del sistema)**

```
$ ./miner 10 4 & sleep 2 && ./miner 10 4
```

```
Miner PID: 142873
Miner PID: 142874
Winner 142873 => [ Y ] => Accepted
Winner 142874 => [ Y ] => Accepted
Winner 142873 => [ Y ] => Accepted
Winner 142874 => [ Y ] => Accepted
Winner 142873 => [ N ] => Rejected
Winner 142873 => [ Y ] => Accepted
Winner 142874 => [ Y ] => Accepted
Winner 142873 => [ Y ] => Accepted
Winner 142874 => [ Y ] => Accepted
Winner 142874 => [ Y ] => Accepted
Winner 142873 => [ Y ] => Accepted
Winner 142874 => [ Y ] => Accepted
Winner 142874 => [ Y ] => Accepted
```

El primer minero arranca y se queda en espera pasiva al estar solo. Dos segundos después entra el segundo minero, completando el mínimo requerido para que el primero despierte y dé comienzo a las rondas de minado. Como el primer minero arrancó antes, su alarma de 10 segundos salta primero y abandona el sistema de forma limpia. El segundo minero, al verse repentinamente solo, pausa dinámicamente el minado y entra en espera pasiva para no desperdiciar CPU, hasta que 2 segundos más tarde expira su propia alarma, destruyendo los semáforos y finalizando el programa con éxito sin dejar bloqueos ni procesos zombis.

- **Mineros con un gran número de hilos**

```
$ ./miner 5 50 & ./miner 5 50 & ./miner 5 50 & ./miner 5 50 & ./miner 5 50
```

```

./miner 5 50 & ./miner 5 50 & ./miner 5 50 & ./miner 5 50 & ./miner 5 50
Miner 145569 added to system
Miner PID: 145569
Miner 145568 added to system
Miner PID: 145569
Miner PID: 145568
Miner 145570 added to system
Miner PID: 145569
Miner PID: 145568
Miner PID: 145570
Miner 145566 added to system
Miner PID: 145569
Miner PID: 145568
Miner PID: 145570
Miner PID: 145566
Miner 145567 added to system
Miner PID: 145569
Miner PID: 145568
Miner PID: 145570
Miner PID: 145566
Miner PID: 145567
Winner 145570 => [ Y Y Y Y ] => Accepted
Winner 145566 => [ Y Y Y Y ] => Accepted
Winner 145567 => [ Y N Y N ] => Accepted
Winner 145570 => [ Y N N Y ] => Accepted
Winner 145568 => [ N Y Y N ] => Accepted
Winner 145566 => [ Y Y N N ] => Accepted
Winner 145566 => [ Y Y N Y ] => Accepted
Winner 145568 => [ Y Y Y N ] => Accepted
Winner 145569 => [ Y Y Y Y ] => Accepted
Winner 145567 => [ Y Y Y Y ] => Accepted
Winner 145568 => [ Y Y Y Y ] => Accepted

```

La ejecución de 5 mineros con 50 hilos cada uno (250 hilos en total) somete al sistema a una prueba de alta concurrencia. Al superar este caso límite garantiza un único ganador ante condiciones de carrera mediante el uso de `sem_trywait`; evita la pérdida de múltiples señales simultáneas (SIGUSR2) gracias a la protección atómica de `sigprocmask` y `sigsuspend`; y previene la corrupción de datos protegiendo el acceso simultáneo a los ficheros compartidos mediante semáforos Mutex.

3. Funcionalidades adicionales

Como mejora de diseño se señalan las siguientes decisiones:

- Semáforo `sem_winner` reutilizable: en vez de destruirlo y recrearlo cada ronda, se libera con `sem_post` al final de la misma, lo que garantiza que solo haya un ganador por ronda sin necesidad de crear/destruir el semáforo repetidamente.
- Fichero `voting.vot` truncado por el ganador: el ganador es el único proceso que abre el fichero en modo escritura ('w') antes de que los votantes escriban, asegurando que los votos de rondas anteriores no contaminen la ronda actual.
- Timeout en `count_votes`: el bucle de recuento del ganador espera como máximo $\text{MAX_RETRIES} \times 100 \text{ ms} = 5 \text{ segundos}$. Esto garantiza que si un votante muere antes de votar, el sistema no se bloquea indefinidamente.
- 20 % de votos erróneos: implementado mediante `rand()%100 < 20` dentro de la función `vote()`, lo que introduce variabilidad realista en el proceso de validación distribuida.

4. Respuesta a las cuestiones del enunciado (Apartado d)

- **¿Se produce algún problema de concurrencia en el sistema descrito?**

Sí. Los principales problemas de concurrencia identificados son:

- Acceso concurrente a miners.log: varios mineros pueden intentar leer y escribir el fichero simultáneamente al registrarse o darse de baja.

- Acceso concurrente a target.tgt: el ganador escribe la nueva solución mientras los perdedores pueden estar leyendo el objetivo actual.

- Acceso concurrente a voting.vot: múltiples perdedores escriben su voto al mismo tiempo.

- Condición de carrera para elegir al ganador: varios procesos pueden haber encontrado la solución casi a la vez y tratar de proclamarse ganadores simultáneamente.

- Pérdida de señales: si una señal SIGUSR1 o SIGUSR2 llega en el momento equivocado (entre la comprobación del flag y la llamada a sigsuspend), el proceso podría quedarse bloqueado indefinidamente esperando una señal que ya llegó.

- **¿Cómo se ha solucionado en el planteamiento dado?**

Cada uno de los problemas anteriores de la siguiente manera:

- Para miners.log se ha usado un semáforo /miners_mutex: todas las lecturas y escrituras sobre este fichero están envueltas en sem_wait/sem_post. La función register_miner() y unregister_miner() son las únicas que acceden al fichero y ambas adquieren el semáforo antes de operar.

- Para target.tgt se ha usado un semáforo /target_mutex: el ganador adquiere sem_target antes de escribir la solución, y cada minero adquiere el mismo semáforo antes de leer el objetivo al inicio de cada ronda, garantizando exclusión mutua.

- Para voting.vot se ha usado un semáforo /vote_mutex: la función vote() hace sem_wait(sem_votes) antes de abrir y escribir en el fichero, y sem_post() inmediatamente

después del `fclose()`. Igualmente, `count_votes()` protege cada lectura del fichero con el mismo semáforo.

- Para la elección del ganador se ha usado un `sem_trywait(sem_winner)`: el semáforo `/winner_mutex` se inicializa a 1 y solo el primer proceso que llama `sem_trywait` con éxito se convierte en ganador. El resto recibe `EINVAL/EAGAIN` y se convierte en votante. El ganador hace `sem_post(sem_winner)` al final de la ronda para que la siguiente funcione correctamente.

- Para la pérdida de señales se ha usado `sigprocmask + sigsuspend`: siguiendo el patrón correcto, la señal se bloquea antes de comprobar el flag y se desbloquea atómicamente durante el `sigsuspend`. Así, si la señal llega entre el bloqueo y el `sigsuspend`, quedará pendiente y se entregará en cuanto `sigsuspend` libere la máscara, evitando el bloqueo indefinido.

El patrón utilizado es el siguiente:

```
sigprocmask(SIG_BLOCK, &block_mask_SIG1, NULL); /* bloquear ANTES */
while (got_sigusr1 == 0)                          /* comprobar flag */
    sigsuspend(&wait_mask);                        /* esperar atómica */
got_sigusr1 = 0;
sigprocmask(SIG_UNBLOCK, &block_mask_SIG1, NULL);
```

- **¿Es sencillo, o siquiera factible, organizar un sistema de este tipo usando únicamente señales?**

No es sencillo, y en la práctica resulta inviable usar exclusivamente señales para implementar este sistema. Los motivos son los siguientes:

- Las señales no pueden transmitir datos: `SIGUSR1` y `SIGUSR2` son notificaciones sin contenido. Para compartir el objetivo de la ronda, la solución encontrada o el resultado de los votos es imprescindible un mecanismo de comunicación adicional (ficheros, memoria compartida, tuberías, etc.).

- Las señales no son acumulables: si un proceso recibe dos señales del mismo tipo antes de procesarlas, el kernel puede entregar solo una. Esto hace imposible garantizar, por ejemplo, que cada votante reciba exactamente una notificación de inicio de votación.

- La exclusión mutua no es posible solo con señales: para garantizar que solo un proceso sea el ganador de la ronda se necesita una operación atómica, que los semáforos proporcionan y las señales no.

- Las señales son asíncronas e impredecibles: coordinar N procesos que compiten, votan y se sincronizan usando solo señales generaría condiciones de carrera imposibles de resolver sin bloqueo/desbloqueo atómico.

En conclusión, las señales son una herramienta adecuada para la notificación asíncrona de eventos (arrancar ronda, detener minado), pero la coordinación completa del sistema requiere combinarlas con semáforos para la exclusión mutua y ficheros para el intercambio de información.

- **¿Cuáles son los mineros que tienden a ganar más monedas?**

Los mineros con mayor número de hilos tienden a ganar más rondas, ya que exploran el espacio de búsqueda más rápidamente. Esta afirmación se verifica con los casos de ejecución recogidos.

Caso 1: ./miner 5 20 & ./miner 5 1 & ./miner 5 1 & ./miner 5 1

Se lanzan cuatro mineros simultáneamente con el mismo tiempo de vida (5 segundos). Uno de ellos dispone de 20 hilos, mientras que los otros tres cuentan únicamente con un hilo cada uno.

El recuento de victorias es el siguiente:

- Minero (20 hilos) = 18 victorias
- Minero (1 hilo) = 1 victoria

- Minero (1 hilo) = 1 victorias
- Minero (1 hilo) = 0 victorias

El minero configurado con más hilos divide el trabajo y procesa el espacio de búsqueda en paralelo, lo que le permite encontrar la solución muchísimo más rápido y ganar casi todas las rondas. Al ser tan veloz, no da tiempo material a que los mineros de un solo hilo terminen sus cálculos, dejándolos siempre en el papel de simples votantes. Las escasas veces que un minero de un solo hilo consigue ganar es por pura probabilidad estadística: da la casualidad de que la respuesta correcta estaba entre los primeros números que le tocó revisar, encontrándola antes de que el minero rápido terminase de barrer su parte.

Caso 2: ./miner 2 30 & ./miner 10 5 & ./miner 10 1

En este escenario se enfrentan dos estrategias distintas: alta potencia a corto plazo frente a menor potencia sostenida en el tiempo. Se lanza un minero con 30 hilos pero una duración de solo 2 segundos, junto a dos mineros de larga duración (10 segundos) con 5 y 1 hilo respectivamente.

El recuento de victorias es el siguiente:

- Minero (30 hilos, 2 s) = 12 victorias
- Minero (5 hilos, 10 s) = 15 victorias
- Minero (1 hilo, 10 s) = 2 victorias

Durante los primeros 2 segundos, el minero de 30 hilos domina el sistema gracias a su enorme capacidad de cálculo en paralelo. Al finalizar su tiempo y desaparecer, el sistema queda exclusivamente en manos de los mineros restantes. En los 8 segundos siguientes, el minero de 5 hilos supera sistemáticamente al de 1 hilo al procesar su búsqueda cinco veces más rápido. Esto demuestra que, aunque un alto número de hilos garantiza una ventaja inmediata, permanecer activo durante más tiempo permite acumular un mayor volumen total de recompensas.

Caso 3: ./miner 8 2 & ./miner 8 4 & ./miner 8 8 & ./miner 8 16

Se ejecutan cuatro mineros simultáneamente con el mismo tiempo de vida (8 segundos), pero con distintos hilos: 2, 4, 8 y 16 hilos.

El recuento de victorias es el siguiente:

- Minero (16 hilos) = 16 victorias
- Minero (8 hilos) = 9 victorias
- Minero (4 hilos) = 5 victorias
- Minero (2 hilos) = 2 victorias

Al disponer todos del mismo tiempo, la probabilidad de encontrar la solución es directamente proporcional al número de hilos asignados. Al duplicar los hilos de un proceso implica que este barre el espacio de búsqueda el doble de rápido, lo que se traduce de forma directa en aproximadamente el doble de victorias frente a sus competidores.

En conclusión, los mineros que ganan más monedas son aquellos configurados con un mayor número de hilos (N_THREADS). Esto ocurre porque el trabajo de buscar la solución se reparte entre todos sus hilos, permitiéndole calcular y encontrar la respuesta mucho más rápido que los demás competidores. Además, el tiempo de vida (N_SECS) también es clave: un minero con mucha potencia (muchos hilos) y que permanezca activo durante más tiempo podrá participar en más rondas, por lo que acumulará siempre muchas más monedas que un minero con pocos hilos o que termine su ejecución rápidamente.